A Mini Project Report Of

PCC-201-COM : Data Structures

Submitted by

Arpita Sandesh Borulkar (63)

Under the Guidance of

Prof. T.H.Patil



Department of Computer Engineering

BHARATI VIDYAPEETH'S
COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115

Savitribai Phule Pune University

[2026-27]

BHARATI VIDYAPEETH
COLLEGE OF ENGINEERING, LAVALE, PUNE

CERTIFICATE

This is to certify that the "**Mini Project Report**" submitted by **Borulkar Arpita Sandesh (63),** on **"Implementation of Palindrome Checker[stack and queue]"** is work done by her and submitted during 2025–2026 academic year, in partial fulfillment of the requirements for the award of the degree of **Bachelor of engineering** in computer Engineering, at **Bharati Vidyapeeth's College of Engineering, Lavale, Pune**.

Prof. T.H.Patil
(Guide)

Dr.Lalit.V.Patil
Head of Computer Engg. Department

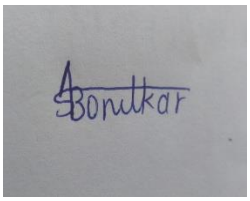
ACKNOWLEDGEMENT

It gives immense pleasure to express our gratitude & thanks to all those who have helped in the timely completion of the Mini-Project work. We have a great pleasure in expressing our deep sense of gratitude and indebtedness to **Prof. T.H.Patil** Guide for their invaluable guidance, constant supervision and sustained encouragement throughout the tenure in this Mini Project work.

We would like to express our thanks to **Dr. Lalit.V.Patil**, Head of Department, Department of Computer Engineering Bharati Vidyapeeth's College of Engineering, Lavale, Pune for their constant support and encouragement.

We would like to express our thanks to **Prof. Dr. R. N. Patil** Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for permitting us to take up the project work. We are thankful to our parents and friends for their kind help and support throughout the course.

Student Name: Arpita Sandesh Borulkar



ABSTRACT

A palindrome checker is a simple application of data structures used to determine whether a given word, number, or text reads the same forward and backward. This mini project presents the implementation of a palindrome checker using Stack and Queue data structures in Python.

The project uses a stack, which follows the Last In First Out (LIFO) principle, and a queue, which follows the First In First Out (FIFO) principle. The input text is inserted into both data structures, and the characters are compared to check whether the given text is a palindrome.

The system accepts input from the user, processes the characters using stack and queue operations, and displays whether the entered text is a palindrome or not. It provides a simple understanding of insertion, deletion, and comparison operations in data structures.

This project demonstrates the practical application of stack and queue concepts and helps improve the understanding of basic data structure operations using Python programming.

Acknowledgement i
Abstract ii
Content iii
List Of Figures iv

TABLE OF CONTENTS

SR.NO.	CONTENT	PAGE NO.
1	Introduction a) Overview b) Problem Statement c) Objective	1
2	Literature Survey	2
3	UML Diagram	3
4	System Requirement	5
5	System Design a) Existing system b) Proposed system c) ER Diagram d) System code	6
6	Result	11
7	Application	13
8	Future Scope	14
9	Conclusion	15
10	References	16

LIST OF FIGURES

Figure	Description	Page No.
Fig 3.1	UML Case Diagram	03
Fig 5.1	System Design	05
Fig 5.2	System Code	07
Fig 5.3	Result	08

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature Survey	2

INTRODUCTION

1.1 Overview:

The project “Implementation of Palindrome Checker Using Stack and Queue” aims to develop a simple program that checks whether a given word or text is a palindrome. A palindrome is a word or sequence that reads the same forward and backward, such as MADAM, LEVEL, and RADAR.

In this project, stack and queue data structures are used to compare characters in reverse and original order. The stack follows the Last In First Out (LIFO) principle, while the queue follows the First In First Out (FIFO) principle. Python programming is used to implement these operations and display the result.

1.2 Objective:

The objective of this project is to design and implement a robust palindrome checker using Stack and Queue data structures that determines whether a given word or text reads the same forward and backward. By utilizing the Last In First Out (LIFO) and First In First Out (FIFO) principles, the system aims to provide accurate results while improving the understanding of fundamental data structure operations.

The system aims to identify palindromes by accepting input from users and comparing characters using Stack and Queue operations. By utilizing these data structures, the palindrome checker will efficiently determine whether the entered text is a palindrome, thereby enhancing the understanding of data structure concepts and their practical applications in computer programming.

LITERATURE SURVEY

Stack and queue are fundamental data structures widely used in computer programming. A stack stores elements according to the Last In First Out (LIFO) principle, whereas a queue follows the First In First Out (FIFO) principle. These structures are useful for processing and organizing data in different applications.

A palindrome checker is a basic application that uses character comparison to determine whether a sequence reads the same forward and backward. Using a stack and a queue together provides a practical demonstration of how different data structures can solve the same problem.

Python provides simple built-in operations and collections that make the implementation of stack and queue operations easy to understand and execute.

Year	Paper Name	Author	Conclusion
1990	"Data Structures and Algorithms"	A. V. Aho, J. E. Hopcroft and J. D. Ullman	Explains fundamental data structures and algorithmic techniques
1998	"The Art of Computer Programming, Volume 1"	Donald E. Knuth	Discusses fundamental algorithms and methods of organizing data.
2009	"Introduction to Algorithms"	Cormen, Leiserson, Rivest and Stein	Provides concepts related to data structures and algorithm analysis.
2013	"Data Structures and Algorithms in Python"	Michael T. Goodrich, Roberto Tamassia and Michael H. Goldwasser	Explains data structures and their implementation in Python.

UML DIAGRAM

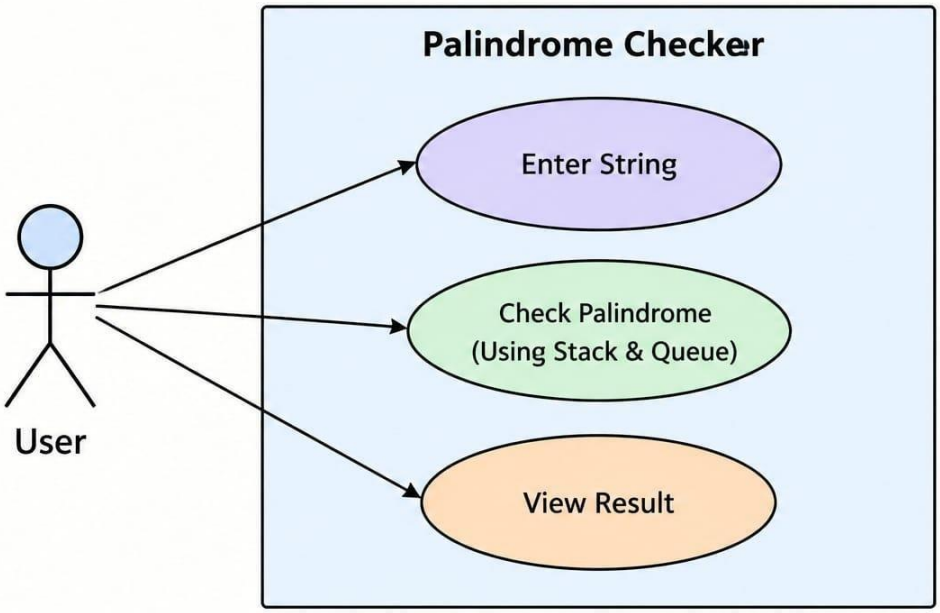


Fig. 3.1: UML Use Case Diagram for Palindrome Checker

SYSTEM REQUIREMENT

- ## HARDWARE REQUIRMENTS

1. Operating System: Windows 11 (64-bit)
2. Processor: Intel Core i3 or higher
3. RAM: Minimum 4 GB
4. Storage: 512 GB or more
5. Processor Speed: 2.30 GHz or higher

- ## SOFTWARE REQUIRMENTS

1. Python 3.x
2. Visual Studio Code (VS Code)
3. Windows Operating System

SYSTEM DESIGN

Existing System:

The existing movie recommendation systems leverage various algorithms and technologies to suggest relevant movies to users based on their preferences and behavior. These systems are implemented across diverse platforms, including streaming services, e-commerce websites, and social media platforms.
Key components of movie recommendation model typically includes:

- ✓ Uses direct string comparison.
- ✓ Relies on built-in reverse functions.
- ✓ Does not demonstrate Stack and Queue operations.
- ✓ Less useful for learning data structure concepts.

Proposed System:

The proposed system implements a palindrome checker using Stack and Queue data structures in Python. The user enters a word or text, and each character is inserted into both data structures. Characters are then removed from the stack and queue one by one and compared. If all characters match, the input is identified as a palindrome; otherwise, it is not.Data Collection and Preprocessing

- ✓ Uses Stack (LIFO) operations.
- ✓ Uses Queue (FIFO) operations.
- ✓ Accepts user input.
- ✓ Compares characters efficiently.
- ✓ Displays whether the input is a palindrome or not.

Entity Relationship Diagram:

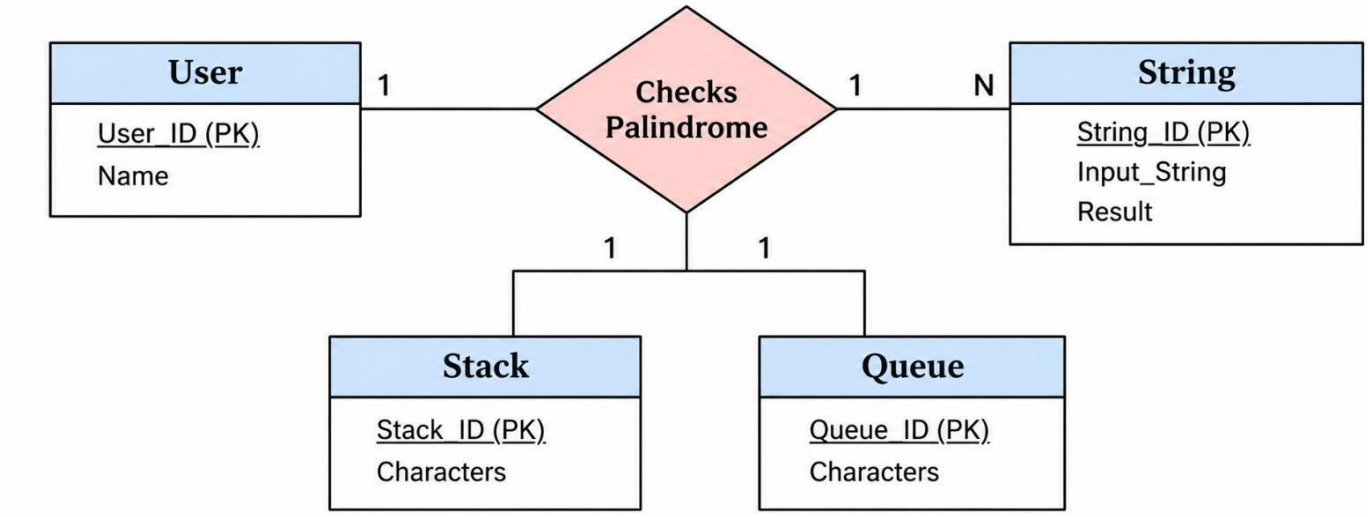


Fig 5.1: ER Diagram

SYSTEM CODE

```
from collections import deque

def palindrome_checker(text):
    # Convert text to lowercase and remove spaces
    text = text.lower().replace(" ", "")

    stack = []
    queue = deque()

    # Insert characters into stack and queue
    for ch in text:
        stack.append(ch)
        queue.append(ch)

    # Compare characters in reverse and original order
    while stack:
        if stack.pop() != queue.popleft():
            print("The given text is NOT a palindrome.")
            return

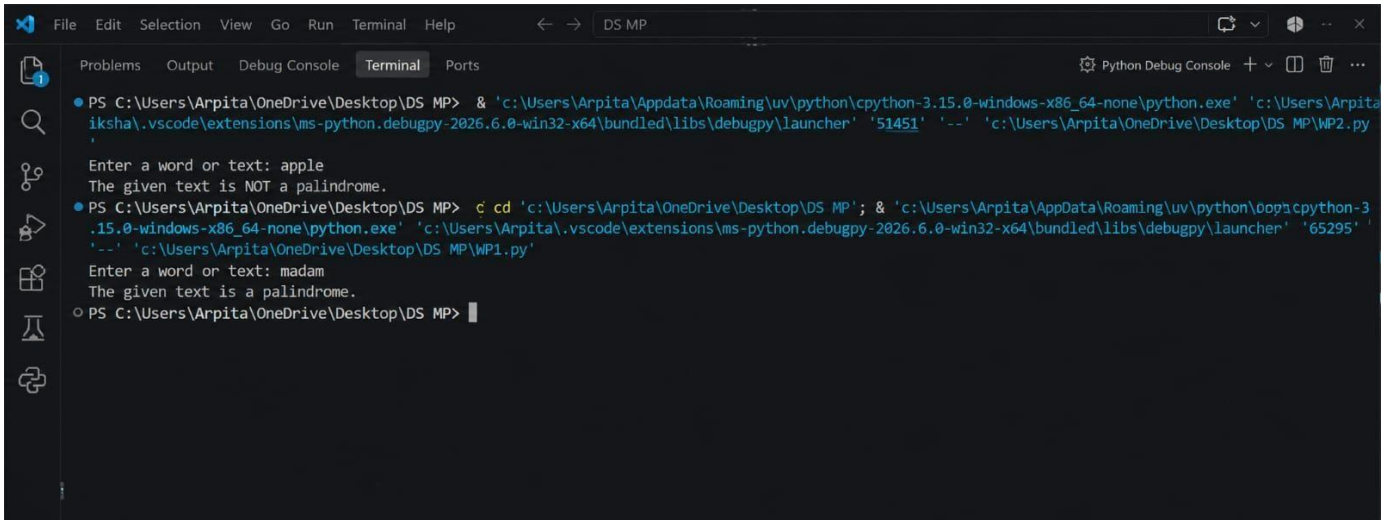
    print("The given text IS a palindrome.")


# Main program
text = input("Enter a word or text: ")
palindrome_checker(text)
```

Fig 5.2: System code

RESULT

5



5.3 result

APPLICATIONS

The applications of the Palindrome Checker project demonstrate the usefulness of Stack and Queue data structures implemented using Python. This project provides practical knowledge of character comparison and text processing, offering value in various programming and educational scenarios.

Educational institutions can use palindrome checker programs to teach students fundamental data structure concepts. By implementing Stack and Queue operations, students can understand the LIFO and FIFO principles and learn how these structures are used to solve programming problems.

Text processing applications can use palindrome checking techniques to identify words or sequences that read the same forward and backward. This can help demonstrate basic string manipulation and character comparison methods in programming.

Software developers can use palindrome checking as a basic programming exercise to understand algorithm development, data organization, and logical problem-solving. It also helps beginners learn how to implement and test algorithms using Python.

Coding practice platforms and programming laboratories can use palindrome checker programs as exercises to improve students' programming skills. These exercises help learners understand conditional statements, loops, Stack and Queue operations, and input processing.

FUTURE WORK

The future of the Palindrome Checker project may involve extending its functionality to support advanced text-processing techniques, such as ignoring spaces, punctuation, and differences between uppercase and lowercase letters. Additional improvements may include checking complete sentences, supporting multiple languages, and developing a graphical user interface (GUI) to make the system more user-friendly.

By exploring these improvements, you can enhance the efficiency, flexibility, and usability of the palindrome checker, ultimately providing a more effective application for learning and implementing Stack and Queue data structures in Python.

CONCLUSION

In this project, we successfully developed a Palindrome Checker using Stack and Queue data structures in Python. The goal was to create a program that can determine whether a given word or text reads the same forward and backward using fundamental data structure operations.

We started by understanding the concepts of Stack and Queue, including the Last In First Out (LIFO) and First In First Out (FIFO) principles. The characters of the input text were processed using both data structures and compared to determine whether the given input was a palindrome. This project helped us understand the practical implementation of data structures and their importance in solving programming problems.

.

REFERENCE

[1] Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1983). *Data Structures and Algorithms.*

[2] Knuth, D. E. (1997). *The Art of Computer Programming, Volume 1: Fundamental Algorithms.*

[3] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms.*

[4] Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data Structures and Algorithms in Python.*

[5] Python Software Foundation. *Python Documentation.* https://docs.python.org/3/